



ckpttn: Multilevel Circuit Partitioning

Exact enumeration first, cheap FM, aggressive coarsening

Wai-Shing Luk

2026



Outline

- 1  Motivation
- 2  Problem Formulation
- 3  Algorithms
- 4  Implementation
- 5  Experimental Results
- 6  Limitations and Threats to Validity
- 7  Conclusion

Section 1



Motivation

Why Circuit Partitioning?

- **Divide and conquer.** Split a netlist into k roughly equal parts.
- **Minimize** the weighted nets that cross between parts.
- It is the first step of physical design and reappears throughout:
 - partitioning-based placement
 - hierarchical design and multi-FPGA mapping
 - load balancing in parallel computing
- Even a few percent of cut reduction matters: cut nets limit **routability** and **timing**.

Four Decades of Partitioning

- **1970** Kernighan–Lin — pairwise interchange as local search
- **1972** Schweikert–Kernighan — the hypergraph (net) model
- **1982** Fiduccia–Mattheyses — single-module moves + bucket queue: one pass is linear
- **1984–98** move-based k -way refinement; spectral methods; network flow
- **1995–99** multilevel — coarsen, partition, uncoarsen + refine (METIS / hMETIS)
- **1998** ISPD98 benchmark suite standardizes how partitioners are compared
- **2016–24** n -level, flow-based, shared-memory tools (KaHyPar, Mt-KaHyPar)
- **2022–24** constraint-driven and learning-based partitioning (TritonPart, SpecPart, GNNs)

ckpttn: Three Principles

- ① **Certify small instances exactly.** A middle-levels Gray-code enumeration visits every balanced bipartition, turning the coarse leaf into a **certified optimum** — the only principle that *guarantees* an optimum.
- ② **Keep FM, but make it cheap.** Most gain updates are **no-ops**; detect and skip them.
- ③ **Coarsen aggressively.** A primal-dual *minimum maximal matching*, with duplicate nets pruned by MinHash.

One algorithm, three ports (**Python** / **C++** / **Rust**), and an explicit **balance warning** that mature tools do not make.

Section 2



Problem Formulation

Balanced Hypergraph Partitioning

Model the circuit as a weighted hypergraph $H = (V, \mathcal{E})$: vertices are modules, hyperedges are nets.

Minimize the cut,

$$\min \sum_{e \in \mathcal{E}} w(e) \mathbb{1}[e \text{ is cut}],$$

subject to a balance constraint, with $W = \sum_{v \in V} m(v)$ and tolerance ϵ :

$$(1 - \epsilon) \frac{W}{k} \leq \sum_{v \in P_i} m(v) \leq (1 + \epsilon) \frac{W}{k}$$

- Tight ϵ (say 1%) makes the problem much harder.
- Some modules may be **fixed** (I/O pads).
- Balanced partitioning is **NP-hard**.

The Choice of Method Tracks the Instance

- **Spectral bisection** — netlists that are close to a mesh
- **Network flow** — instance whose balance can be relaxed
- **Exact enumeration** — up to a few tens of modules
- **Multilevel** — tight tolerance, large instances
- **Flat FM** — loose tolerance

ckpttn exposes the extremes of this map through its **presets**.

Section 3



Algorithms



Algorithms

*We begin with the exact method, because it is the only one that **certifies** optimality. It is practical only on small graphs; the local-search refiners and the multilevel framework that follow make large instances tractable.*

Exhaustive: Middle-Levels Gray Code

- Give each module one bit (0 = block A, 1 = block B); a balanced bipartition has n or $n + 1$ ones.
- The **middle-levels graph** G_n of the $(2n + 1)$ -cube has all bitstrings of weight n or $n + 1$ as vertices, with edges between strings differing in one bit.
- A **Hamiltonian cycle** visits every balanced bipartition exactly once.
- Each transition is **one module flip** — a single FM move.
- Conjectured in 1982, proved constructively only recently (Gregor–Mütze–Nummenpalo).
- Cost per step is $O(\deg)$; the visit count is $2^{\binom{2n+1}{n}}$.

The Exact Leaf Is Cheap Only When It Is Small

Modules n	States	Time
11	924	< 1 ms
15	12,870	< 1 ms
20	705,432	0.20 s
25	10.4 M	3.4 s

The enumeration is kept to the **coarsest** level (a few tens of modules), where it certifies the optimum; the uncoarsening then refines heuristically.

FM: Gain and the Bucket Queue

- Gain of moving v = reduction in cut:

$$g(v) = FS(v) - TE(v),$$

where FS counts nets entirely in v 's block and TE nets entirely in the destination.

- Cuts are kept in a **bucket** priority queue keyed by integer gain: insert / extract-max / key-update in $O(1)$ amortized.
- One **pass**:
 - ① initialize gains, build buckets;
 - ② repeatedly take the highest-gain legal move, **lock** the module, update its neighbours' gains;
 - ③ when no legal move remains, restore the best partition seen.

FM: The No-Op That Pays

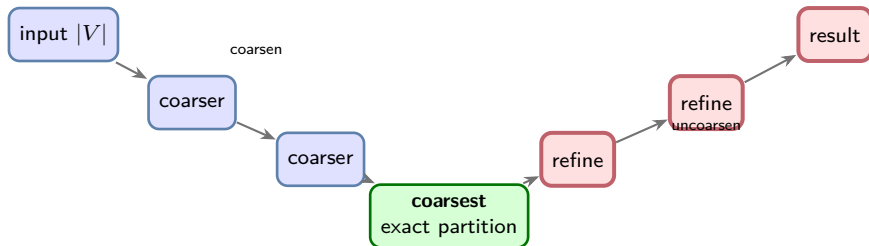
- For a general net with ≥ 2 pins on **each** side, moving one pin leaves every neighbour's gain unchanged.
- Skip the update with a single test instead of a queue operation.
- In the Python reference this cuts `modify_key` calls on `ibm03` from **7.37 M** to **374 K**.

The single most profitable optimization in the whole partitioner.

NN: A Cheaper Greedy Refiner

- Drop the snapshot/rollback/locking machinery — pure greedy descent.
- Take the highest-gain move while its gain is positive.
- Monotone descent $\Rightarrow$ terminates at a local optimum; no locking needed.
- **8–35 $\times$ faster**, but **1.4–2.2 $\times$ worse** cut on flat graphs.
- Inside the multilevel loop it recovers much of FM's quality — exactly the role of a cheap refiner.

Multilevel: Coarsen $\rightarrow$ Partition $\rightarrow$ Refine



- **Coarsen** with a min-maximal matching; merge duplicate nets (MinHash).
- **Initial partition** by exact enumeration on the coarsest graph.
- **Uncoarsen + refine** with FM or NN at every level.
- Recursion guard: contract only if $|V^+| \cdot 1.5 < |V|$.

K-Way: Pairwise Refinement

- For $k > 2$, optimize each pair of blocks (i, j) in turn.
- Movable set = modules in i or j ; everything else is fixed.
- Run the exact leaf on the pair with the **full netlist**, so cross-block pins are counted correctly.
- Leaf budget $25k/2$ movable modules spread over $\binom{k}{2}$ pairs.

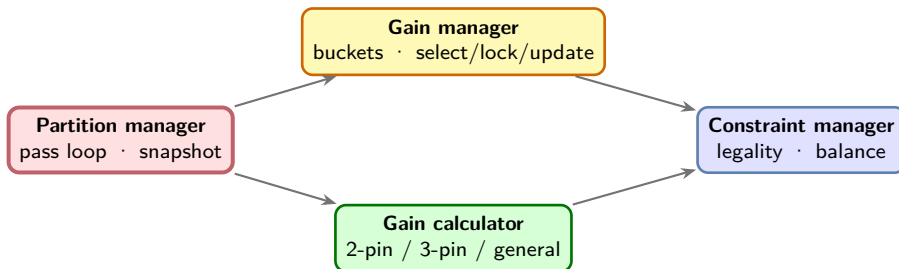
k	2	3	4	8
Time (s)	4.25	16.2	15.9	413

Section 4



Implementation

Four Collaborating Objects



- **Template Method** fixes the pass skeleton (select, legalize, apply, update).
- **Strategy** lets the gain, constraint, and partition managers be swapped.
- One gain calculator serves the **flat**, **multilevel**, and **k-way** partitioners.

Three Ports, One Specification

- **Python** (ckpttnpy, NetworkX) · **C++** (ckpttn-cpp, templates) · **Rust** (ckpttn-rs, petgraph).
- All three share a **SplitMix64** seed stream: a seed denotes the same initial partition everywhere.
- Cross-language regression tests compare the resulting cut.

```
ckpttnpy circuit.hgr <k> <epsilon>
```

Presets bundle the balance tolerance and the mode (default 3%, quality 1%, highest_quality 0.5%).

Section 5



Experimental Results

Refiners: FM vs NN (p1 and ibm03, $k = 2$, $\epsilon = 3\%$)

Benchmark	Refiner	Multilevel	Mean cut	Median time (s)
p1	FM	no	105.4	0.23
p1	FM	yes	77.4	1.01
p1	NN	no	231.4	0.03
p1	NN	yes	106.0	0.80
ibm03	FM	no	4021.2	10.31
ibm03	FM	yes	1840.2	44.69
ibm03	NN	no	7453.8	0.76
ibm03	NN	yes	4081.0	52.99

FM dominates quality; NN dominates speed; multilevel helps **both**, and helps NN far more.

Multilevel versus Flat FM (*ibm01*, $k = 2$, $\epsilon = 3\%$)

Metric	FM-only	Multilevel	Change
Mean cut	1377.6	772.4	−43.9%
Best cut	949	385	−59.4%
Worst cut	1830	1192	−34.9%
Mean time (s)	6.59	19.49	2.96×

Multilevel lowers the cut on **every** run and shrinks the spread ($345 \rightarrow 278$), at about three times the runtime.

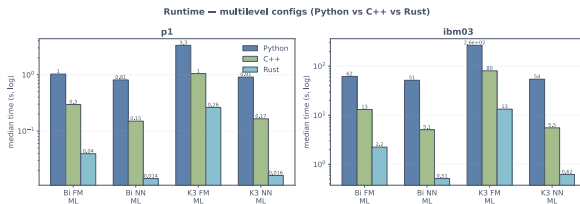
Cross-Language Runtime (release builds)

Benchmark	C++ (ms)	Rust (ms)	Python (ms)	C++/Python
p1 (833 modules)	3.42	51.2	202	5
ibm03 (23,136 modules)	262	1467	9691	3

The compiled ports are **one to two orders of magnitude** faster than the interpreted one — the algorithm, not the language, is the contribution.

Scaling to 210k Modules

- Full **ISPD98** suite, $k = 2$, $\epsilon = 3\%$, one start (C++ port).
- 0.52 s on ibm01 (12,752 modules) $\rightarrow$ 19.1 s on ibm18 (210,613).
- Peak resident memory 227 MB (69k) $\rightarrow$ 557 MB (211k).
- Runtime grows **near-linearly** with size.



Single-Threaded Head-to-Head ($\epsilon = 3\%$, $k = 2$)

Instance	Tool	Cut	Time (s)	Imbalance	Balanced
ibm01	ckpttn	973	0.52	0.86%	Yes
ibm01	hMetis	240	0.13	4.82%	No
ibm01	Mt-KaHyPar	212	0.28	2.54%	Yes
ibm03	ckpttn	2169	0.92	0.83%	Yes
ibm03	hMetis	1012	0.32	0.72%	Yes
ibm03	Mt-KaHyPar	993	0.64	0.45%	Yes
ibm10	ckpttn	4027	3.70	0.94%	Yes
ibm10	hMetis	1276	1.01	3.87%	No
ibm10	Mt-KaHyPar	1509	1.44	< 0.01%	Yes

- ckpttn: the **worst cut and the slowest**, but the **best balance** ($< 1\%$).
- hMetis **silently violates** the requested 3% bound on ibm01 and ibm10.
- No parity with the state of the art: the contribution is **reproducible ports and honest diagnostics**.

Section 6



Limitations and Threats to Validity

What We Must State Plainly

- **Balance under tight tolerance** — the lower bound was $\text{round}(2W\epsilon/k)$ instead of $(1 - \epsilon)W/k$, so a block could shrink to ϵW (p1: 54/779). Fixed in all three ports (p1 now 406/427).
- **Scale** — tested to 210k modules; no million-gate designs (e.g. Titan23).
- **Head-to-head** — one run per tool on three instances; ckpttn's cut is $2\text{--}5\times$ larger than both baselines.
- **Exact leaf** — exponential, ≤ 25 coarse modules; bounded by the coarsening loss.
- **k -way** — $\binom{k}{2}$ pairwise sweeps scale poorly ($k = 8$: 413 s).
- **Single-threaded core** — only the multi-start loop is parallel.
- **Benchmark comparability** — the free ISPD98 hgr omits the fixed-I/O-pad convention, so its cut is not the published leaderboard's.

Section 7



Conclusion

What We Learned

- A **classical** partitioner can be both **simple and trustworthy**.
- Order matters: **exact first**, then a cheap FM, then aggressive coarsening.
- The three parts **agree** because Template Method + Strategy separate the policies from the pass loop.
- The **explicit balance warning** removes the silent-failure mode that complicates automated tuning and reproducibility.

Next Steps

- A flat (single-array) graph layout in the Rust port.
- A **parallel** FM refinement.
- **Timing-aware** refinement, in the spirit of TritonPart.
- A systematic study of the contraction guard and leaf threshold across the IBM family.



Thank You

Thank You!

Questions?